

Sistema de Gestão de Territórios e Publicações (GTP)

Documento 08 – Segurança

Projeto: Sistema de Gestão de Territórios e Publicações (GTP)

Versão: 1.0.0

Status: Em Elaboração

Data: Julho/2026

Autor: Fabio André Zatta

1 – Introdução

1.1 Objetivo

Este documento define a arquitetura de segurança do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é estabelecer as diretrizes, mecanismos, políticas e controles de segurança que deverão ser adotados durante o desenvolvimento, implantação e operação do sistema, garantindo a proteção das informações, dos usuários e da infraestrutura.

A arquitetura de segurança foi concebida considerando as boas práticas atuais de desenvolvimento seguro, autenticação moderna, proteção contra ataques comuns em aplicações web e rastreabilidade das operações executadas.

As definições apresentadas complementam os seguintes documentos do projeto:

- Documento 02 – Requisitos Funcionais;
- Documento 03 – Regras de Negócio;
- Documento 04 – Casos de Uso;
- Documento 05 – Arquitetura do Sistema;
- Documento 06 – Arquitetura Backend;
- Documento 07 – Arquitetura Frontend;
- Documento 10 – API REST.

1.2 Escopo

Este documento contempla a segurança de toda a solução GTP.

Inclui:

- autenticação;
- autorização;
- gerenciamento de sessões;
- segurança da API;
- segurança do frontend;
- segurança do backend;
- proteção dos dados;
- criptografia;
- auditoria;
- rastreabilidade;

- segurança da infraestrutura;
- segurança em Docker;
- segurança do banco de dados;
- monitoramento;
- boas práticas de desenvolvimento seguro.

Não fazem parte deste documento políticas organizacionais, gestão de equipamentos físicos ou processos administrativos externos ao sistema.

1.3 Objetivos da Arquitetura de Segurança

A arquitetura de segurança possui os seguintes objetivos:

- garantir confidencialidade das informações;
- preservar a integridade dos dados;
- assegurar disponibilidade dos serviços;
- controlar rigorosamente os acessos;
- impedir operações não autorizadas;
- permitir rastreabilidade completa das ações;
- reduzir riscos operacionais;
- facilitar auditorias;
- proteger dados sensíveis;
- apoiar a evolução segura da aplicação.

1.4 Princípios de Segurança

Toda a solução será construída observando os seguintes princípios.

Defesa em Profundidade (Defense in Depth)

A proteção será aplicada em múltiplas camadas.

Exemplos:

- frontend;
- API;
- autenticação;
- autorização;
- banco de dados;
- infraestrutura;
- rede;

- monitoramento.

Nenhuma camada deverá ser considerada suficiente isoladamente.

Menor Privilégio (Least Privilege)

Todo usuário deverá possuir apenas as permissões necessárias para executar suas atividades.

O sistema evitará concessões excessivas de acesso.

Falha Segura (Fail Secure)

Na ocorrência de falhas, o sistema deverá adotar um comportamento seguro.

Exemplos:

- negar acesso quando houver dúvida sobre permissões;
- encerrar sessões inválidas;
- bloquear operações inconsistentes.

Separação de Responsabilidades

A autenticação, autorização, regras de negócio e persistência permanecerão desacopladas.

Cada componente possuirá responsabilidades bem definidas.

Validação em Todas as Camadas

Toda informação recebida será validada:

- no frontend;
- na API;
- na camada de aplicação;
- na camada de domínio;
- na persistência.

Essa abordagem reduz riscos decorrentes de dados malformados ou manipulados.

1.5 Pilares da Segurança da Informação

A arquitetura seguirá os pilares clássicos da segurança da informação.

Pilar	Aplicação no GTP
-------	------------------

Confidencialidade	Controle rigoroso de acesso aos dados.
-------------------	--

Integridade	Validação, auditoria e proteção contra alterações indevidas.
-------------	--

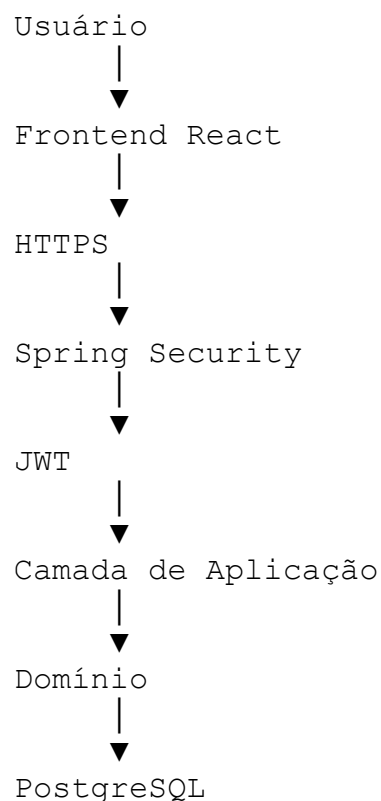
Disponibilidade	Arquitetura preparada para recuperação de falhas.
-----------------	---

Autenticidade	Identificação confiável dos usuários.
---------------	---------------------------------------

Rastreabilidade	Registro das operações críticas realizadas no sistema.
-----------------	--

1.6 Arquitetura Geral da Segurança

A segurança será aplicada de forma transversal em toda a solução.



Cada camada adiciona mecanismos específicos de proteção.

1.7 Ameaças Consideradas

A arquitetura foi planejada para mitigar riscos como:

- acesso não autorizado;

- roubo de credenciais;
- sequestro de sessão;
- manipulação de parâmetros;
- força bruta;
- exposição de informações sensíveis;
- ataques de injeção;
- Cross-Site Scripting (XSS);
- Cross-Site Request Forgery (CSRF), quando aplicável;
- vazamento de dados;
- elevação indevida de privilégios;
- exploração de dependências vulneráveis.

1.8 Tecnologias de Segurança

O projeto utilizará tecnologias consolidadas no ecossistema Java e React.

Backend

- Spring Security
- JWT
- BCrypt
- Jakarta Validation
- HTTPS/TLS
- Flyway
- PostgreSQL

Frontend

- React 19
- TypeScript
- Axios
- TanStack Query
- Zustand
- React Router

Infraestrutura

- Docker
- Docker Compose
- Nginx (quando utilizado)

- Certificados TLS
- Variáveis de ambiente

1.9 Relação com os Demais Documentos

Este documento complementa:

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a visão arquitetural geral.
Documento 06 – Arquitetura Backend	Implementa os mecanismos de segurança no servidor.
Documento 07 – Arquitetura Frontend	Define autenticação e proteção da interface.
Documento 09 – Docker	Trata da segurança do ambiente de execução.
Documento 10 – API REST	Especifica autenticação e autorização dos endpoints.

1.10 Estrutura do Documento

O Documento 08 será organizado nos seguintes capítulos:

1. Introdução
2. Arquitetura Geral da Segurança
3. Autenticação
4. Autorização e Controle de Acesso
5. Segurança da API REST
6. Segurança do Frontend
7. Segurança do Backend
8. Criptografia e Proteção de Dados
9. Auditoria e Logs
10. Segurança da Infraestrutura
11. Segurança em Docker
12. Segurança do Banco de Dados
13. Monitoramento e Resposta a Incidentes
14. Boas Práticas de Desenvolvimento Seguro
15. Conclusão

2 – Arquitetura Geral da Segurança

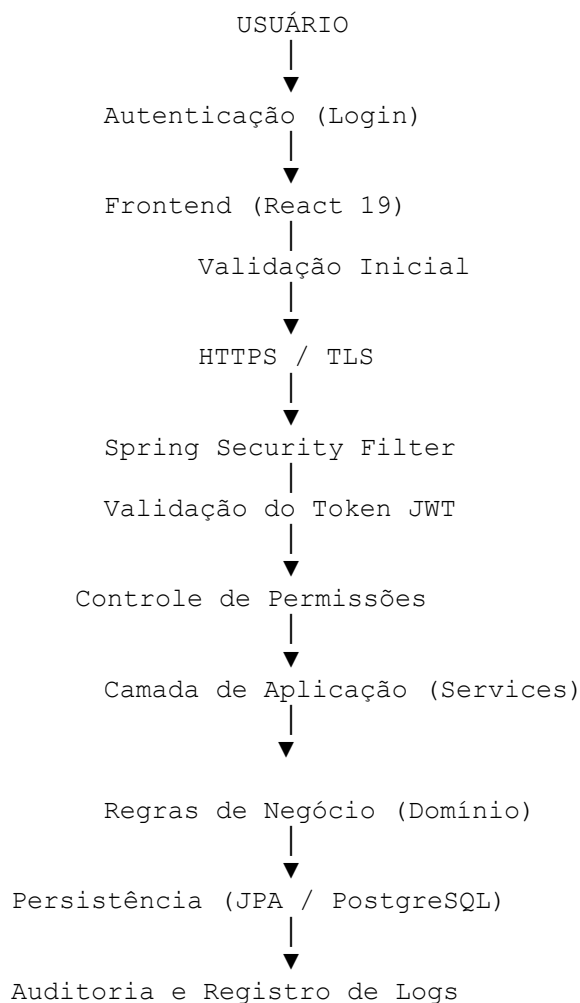
2.1 Objetivo

Este capítulo descreve a arquitetura geral de segurança do **Gestor de Territórios e Publicações (GTP)**, apresentando os princípios, camadas e mecanismos responsáveis por proteger a aplicação contra acessos indevidos, manipulação de dados, vulnerabilidades conhecidas e ameaças operacionais.

A arquitetura foi projetada para garantir que a segurança seja tratada como um aspecto transversal da solução, presente desde a interface do usuário até a persistência dos dados.

2.2 Visão Geral da Arquitetura

A segurança do GTP será estruturada em múltiplas camadas independentes e complementares.



Cada camada adiciona mecanismos específicos de proteção e não depende exclusivamente das demais para garantir a segurança.

2.3 Modelo de Defesa em Camadas (Defense in Depth)

O GTP adotará o princípio de **Defesa em Profundidade (Defense in Depth)**.

Esse modelo considera que nenhuma camada de proteção é suficiente isoladamente.

Assim, várias barreiras independentes são combinadas para reduzir o impacto de falhas ou ataques.

As camadas de proteção incluem:

1. Interface do usuário (Frontend);
2. Comunicação segura;
3. API REST;
4. Autenticação;
5. Autorização;
6. Regras de negócio;
7. Persistência;
8. Banco de dados;
9. Infraestrutura;
10. Monitoramento e auditoria.

Caso uma camada seja comprometida, as demais continuam oferecendo proteção.

2.4 Arquitetura Zero Trust

A arquitetura seguirá os princípios do modelo **Zero Trust**, segundo o qual nenhuma requisição deve ser considerada confiável por padrão.

Princípios adotados:

- verificar toda identidade;
- validar todas as requisições;
- autenticar continuamente;
- autorizar cada operação;
- limitar privilégios;
- monitorar todas as ações relevantes.

Mesmo usuários autenticados deverão ter suas permissões verificadas a cada operação.

2.5 Separação de Responsabilidades

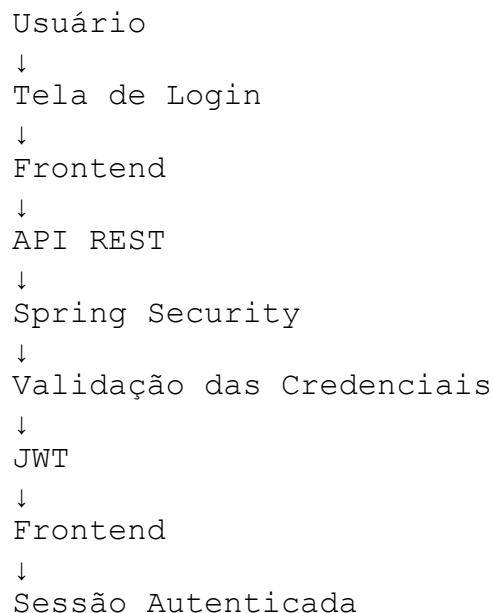
Os mecanismos de segurança serão distribuídos entre diferentes componentes da arquitetura.

Camada	Responsabilidade
Frontend	Coleta de credenciais, proteção de rotas e gerenciamento da sessão.
API REST	Recepção das requisições e encaminhamento para autenticação e autorização.
Spring Security	Autenticação, validação do JWT e controle de acesso.
Aplicação	Execução dos casos de uso autorizados.
Domínio	Aplicação das regras de negócio.
Persistência	Controle das operações sobre os dados.
Banco de Dados	Armazenamento seguro e integridade das informações.

Essa separação reduz o acoplamento e facilita a manutenção da segurança.

2.6 Fluxo Geral de Autenticação

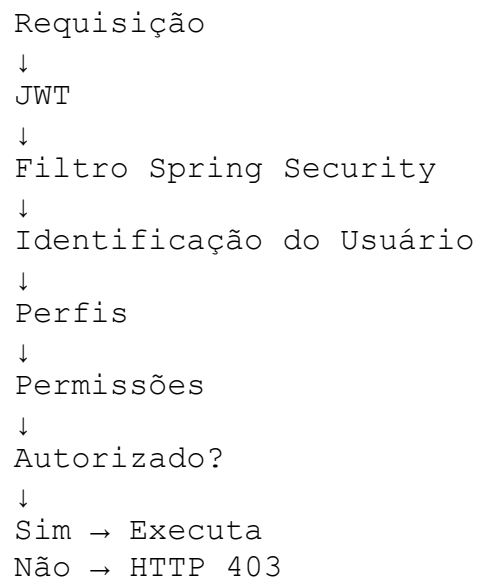
O processo de autenticação seguirá o fluxo abaixo.



Após a autenticação, o token JWT será utilizado para identificar o usuário nas requisições subsequentes.

2.7 Fluxo de Autorização

A autorização ocorrerá em todas as operações protegidas.



Esse fluxo garante que apenas usuários autorizados executem operações específicas.

2.8 Camadas de Proteção

A arquitetura de segurança será organizada em camadas complementares.

Camada 1 – Interface

Responsável por:

- proteção de rotas;
- gerenciamento da sessão;
- ocultação de funcionalidades não autorizadas;
- validação preliminar dos dados.

Camada 2 – Comunicação

Responsável por:

- HTTPS obrigatório;
- criptografia da comunicação;
- validação de certificados;
- prevenção contra interceptação de dados.

Camada 3 – API

Responsável por:

- autenticação;
- autorização;
- validação das requisições;
- tratamento de erros;
- limitação de acesso.

Camada 4 – Domínio

Responsável por:

- regras de negócio;
- validações críticas;
- consistência das operações.

Camada 5 – Persistência

Responsável por:

- integridade dos dados;
- controle transacional;
- proteção contra operações inválidas.

Camada 6 – Auditoria

Responsável por:

- rastrear operações;
- registrar eventos;
- identificar alterações;
- apoiar investigações.

2.9 Controle de Fronteiras

Cada camada validará apenas aquilo que lhe compete.

Exemplo:

Frontend:

- obrigatoriedade dos campos;

- formatos;
- experiência do usuário.

Backend:

- autenticação;
- autorização;
- regras de negócio;
- consistência.

Banco:

- integridade referencial;
- restrições;
- transações.

Essa divisão evita redundâncias e melhora a organização do sistema.

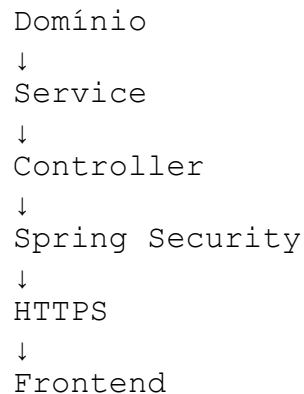
2.10 Fluxo Seguro de Requisições

```
Frontend
↓
Axios
↓
HTTPS
↓
Spring Security
↓
JWT Filter
↓
Controller
↓
Service
↓
Domínio
↓
Repository
↓
PostgreSQL
```

Cada etapa realiza validações antes que a requisição prossiga.

2.11 Fluxo Seguro de Respostas

O retorno da API seguirá um fluxo controlado.



Durante esse processo:

- informações sensíveis serão omitidas;
- erros internos não serão expostos;
- mensagens serão padronizadas.

2.12 Integração entre Frontend e Backend

A segurança dependerá da colaboração entre ambas as camadas.

Frontend:

- envia credenciais;
- armazena a sessão;
- protege a navegação;
- envia o JWT.

Backend:

- autentica;
- autoriza;
- aplica regras;
- registra auditoria;
- responde às requisições.

2.13 Gestão da Sessão

A sessão do usuário será controlada por:

- autenticação inicial;

- token JWT;
- tempo de expiração;
- encerramento explícito;
- invalidação em caso de falhas de segurança.

2.14 Tratamento de Incidentes

A arquitetura deverá prever respostas para situações como:

- token inválido;
- sessão expirada;
- tentativa de acesso indevido;
- repetidas falhas de autenticação;
- exceções inesperadas;
- indisponibilidade temporária.

Cada situação possuirá tratamento específico e registro apropriado.

2.15 Observabilidade da Segurança

Os mecanismos de segurança produzirão informações para monitoramento.

Eventos registrados incluem:

- logins;
- logouts;
- alterações de permissões;
- falhas de autenticação;
- acessos negados;
- operações críticas;
- erros de validação;
- exceções relevantes.

Esses dados apoiarão auditorias e investigações.

2.16 Escalabilidade da Arquitetura

A arquitetura foi projetada para suportar:

- novos perfis de usuário;
- autenticação multifator (MFA);

- provedores externos de identidade;
- múltiplas congregações;
- ambientes distribuídos;
- novos módulos do sistema.

Essas expansões poderão ser incorporadas sem necessidade de reformular a arquitetura principal.

2.17 Benefícios da Arquitetura

A arquitetura proposta oferece:

- proteção em múltiplas camadas;
- baixo acoplamento;
- validações distribuídas;
- maior rastreabilidade;
- facilidade para auditoria;
- integração consistente entre frontend e backend;
- preparação para crescimento futuro.

2.18 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a estrutura arquitetural da solução.
Documento 06 – Arquitetura Backend	Implementa os mecanismos de segurança do servidor.
Documento 07 – Arquitetura Frontend	Define a proteção da interface e da navegação.
Documento 09 – Docker	Estabelece a segurança do ambiente de execução.
Documento 10 – API REST	Especifica autenticação, autorização e contratos dos serviços.

3 – Autenticação

3.1 Objetivo

Este capítulo define a arquitetura de autenticação do **Gestor de Territórios e Publicações (GTP)**, estabelecendo os mecanismos responsáveis por verificar a identidade dos usuários antes da concessão de acesso aos recursos protegidos da aplicação.

A autenticação será baseada em padrões amplamente adotados pela indústria, utilizando **Spring Security, JSON Web Token (JWT)** e comunicação segura via **HTTPS/TLS**.

Seu objetivo é garantir que apenas usuários devidamente identificados possam iniciar sessões no sistema, reduzindo riscos de acesso indevido e preservando a integridade das operações.

3.2 Princípios da Autenticação

A estratégia de autenticação seguirá os seguintes princípios:

- autenticação centralizada;
- credenciais protegidas;
- comunicação criptografada;
- sessões stateless;
- tokens assinados digitalmente;
- expiração controlada das sessões;
- proteção contra reutilização indevida de credenciais;
- integração transparente entre frontend e backend.

Esses princípios asseguram um modelo de autenticação seguro e de fácil manutenção.

3.3 Componentes Envolvidos

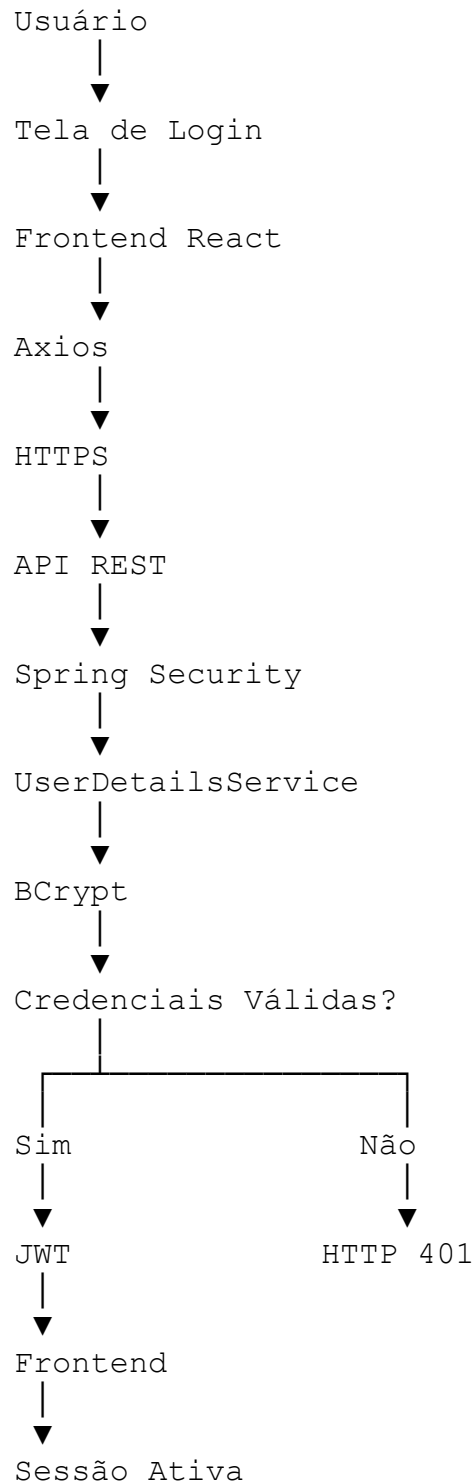
A autenticação envolve diferentes componentes da arquitetura.

Componente	Responsabilidade
Frontend (React)	Coletar credenciais e iniciar a autenticação.
Axios	Enviar as requisições de login.
API REST	Receber e encaminhar as credenciais.
Spring Security	Validar o usuário e emitir o JWT.
Serviço de Usuários	Recuperar informações do usuário.
BCrypt	Verificar a senha criptografada.
JWT Provider	Gerar e validar tokens.
PostgreSQL	Armazenar credenciais e perfis.

Cada componente possui responsabilidades bem definidas, favorecendo a separação de responsabilidades.

3.4 Fluxo Geral de Autenticação

O processo de autenticação seguirá o fluxo abaixo.



3.5 Processo de Login

O login compreenderá as seguintes etapas:

1. O usuário informa seu identificador e senha.
2. O frontend realiza validações básicas do formulário.
3. As credenciais são enviadas via HTTPS.
4. O Spring Security autentica o usuário.
5. A senha é comparada utilizando BCrypt.
6. O backend verifica se a conta está ativa.
7. Em caso de sucesso, um JWT é emitido.
8. O frontend armazena a sessão.
9. O usuário é redirecionado para o Dashboard.

Caso a autenticação falhe, nenhuma informação sensível será revelada.

3.6 Identificação do Usuário

O sistema permitirá a autenticação utilizando um identificador único definido pelo domínio da aplicação.

Esse identificador poderá ser:

- nome de usuário;
- endereço de e-mail (caso habilitado futuramente);
- outro identificador corporativo.

Independentemente do identificador adotado, o sistema garantirá sua unicidade.

3.7 Armazenamento de Senhas

As senhas **jamais serão armazenadas em texto puro.**

Diretrizes:

- utilização do algoritmo **BCrypt**;
- salt gerado automaticamente;
- fator de custo configurável;
- impossibilidade de recuperação da senha original.

Em caso de redefinição, uma nova senha será gerada pelo usuário.

3.8 JSON Web Token (JWT)

Após a autenticação, será emitido um JWT contendo as informações necessárias para identificação da sessão.

O token poderá incluir:

- identificador do usuário;
- perfis;
- permissões;
- congregação ativa;
- data de emissão;
- data de expiração.

Informações sensíveis, como senhas ou dados pessoais desnecessários, **não deverão ser incluídas no token.**

3.9 Características do JWT

O token deverá possuir:

- assinatura digital;
- tempo de expiração configurável;
- integridade garantida;
- impossibilidade de alteração sem invalidação da assinatura.

A validação será realizada pelo Spring Security em todas as requisições protegidas.

3.10 Sessões Stateless

A autenticação será baseada em sessões **stateless**.

Características:

- o servidor não manterá estado da sessão;
- cada requisição deverá conter um JWT válido;
- a autenticação será reavaliada a cada requisição.

Essa abordagem favorece escalabilidade e simplifica ambientes distribuídos.

3.11 Armazenamento da Sessão no Frontend

O frontend será responsável por manter apenas as informações necessárias ao funcionamento da aplicação.

Boas práticas:

- evitar persistência de dados sensíveis;
- limpar informações ao realizar logout;

- impedir exposição do token em logs;
- centralizar o gerenciamento da sessão na camada de autenticação.

A estratégia específica de armazenamento (memória, cookies seguros ou outro mecanismo) será definida conforme os requisitos de implantação e detalhada em documento complementar.

3.12 Expiração da Sessão

Toda sessão deverá possuir tempo de validade.

Quando expirada:

- o token será considerado inválido;
- novas requisições autenticadas serão rejeitadas;
- o frontend encerrará a sessão;
- o usuário será redirecionado para a tela de login.

3.13 Renovação de Sessão

A arquitetura poderá ser estendida para suportar renovação controlada da sessão.

Essa funcionalidade poderá utilizar:

- refresh token;
- renovação silenciosa;
- políticas de expiração configuráveis.

Sua adoção dependerá das necessidades operacionais e dos requisitos de segurança.

3.14 Logout

O encerramento da sessão compreenderá:

1. remoção do token da aplicação;
2. limpeza do estado de autenticação;
3. invalidação de caches relacionados ao usuário;
4. redirecionamento para a tela de login.

Caso o backend implemente mecanismos de revogação de tokens, estes também deverão ser acionados.

3.15 Tratamento de Falhas de Autenticação

As seguintes situações deverão ser tratadas de forma uniforme:

- credenciais inválidas;
- usuário inexistente;
- conta desativada;
- conta bloqueada;
- senha expirada (quando aplicável);
- token inválido;
- token expirado.

As mensagens retornadas ao usuário deverão ser genéricas, evitando fornecer informações que possam facilitar ataques.

3.16 Proteção contra Ataques de Força Bruta

A arquitetura deverá prever mecanismos para reduzir tentativas automatizadas de autenticação.

Exemplos:

- limitação de tentativas consecutivas;
- bloqueio temporário da conta após sucessivas falhas;
- registro de tentativas suspeitas;
- monitoramento de padrões anômalos.

Esses mecanismos serão configuráveis conforme a política de segurança da aplicação.

3.17 Integração com Autorização

A autenticação apenas identifica o usuário.

Após a autenticação, o processo de autorização verificará:

- perfis;
- permissões;
- acesso às funcionalidades;
- restrições de operação.

Essa separação mantém responsabilidades claras entre identificação e controle de acesso.

3.18 Auditoria da Autenticação

Os seguintes eventos deverão ser registrados:

- login realizado com sucesso;
- falhas de autenticação;
- logout;
- bloqueios de conta;
- redefinição de senha;
- expiração de sessão;
- tentativas de acesso com token inválido.

Esses registros apoiarão auditorias e investigações de incidentes.

3.19 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Implementa o Spring Security e a emissão do JWT.
Documento 07 – Arquitetura Frontend	Gerencia a sessão e protege as rotas da interface.
Documento 10 – API REST	Define os endpoints de autenticação e seus contratos.

4 – Autorização e Controle de Acesso

4.1 Objetivo

Este capítulo estabelece a arquitetura de autorização e controle de acesso do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é definir os mecanismos responsáveis por determinar quais funcionalidades, recursos e operações poderão ser executados por cada usuário autenticado, garantindo que o acesso seja concedido apenas quando houver permissão explícita.

A autorização será implementada de forma centralizada no backend, utilizando **Spring Security**, enquanto o frontend adaptará a interface conforme os perfis e permissões recebidos.

4.2 Conceitos Fundamentais

A autorização será baseada em três conceitos principais:

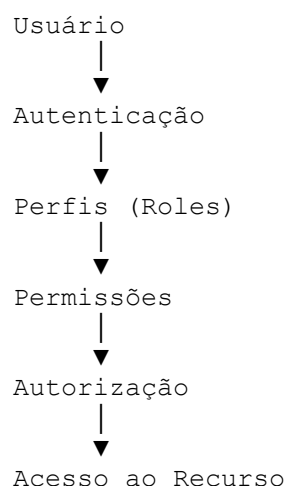
- **Usuário:** identidade autenticada que acessa o sistema.
- **Perfil (Role):** conjunto de responsabilidades atribuídas ao usuário.
- **Permissão (Permission):** autorização específica para executar determinada operação.

Essa separação proporciona flexibilidade para atender diferentes cenários de uso sem duplicação de regras.

4.3 Modelo de Controle de Acesso

O GTP adotará o modelo **RBAC (Role-Based Access Control)**, no qual o acesso é concedido conforme os perfis atribuídos ao usuário.

Fluxo simplificado:



Esse modelo facilita a administração de usuários e reduz a complexidade da gestão de permissões.

4.4 Princípio do Menor Privilégio

O sistema seguirá rigorosamente o princípio do **Menor Privilégio (Least Privilege)**.

Cada usuário receberá apenas as permissões necessárias para desempenhar suas funções.

Benefícios:

- redução da superfície de ataque;
- menor risco de uso indevido;
- simplificação da auditoria;
- maior segurança operacional.

4.5 Perfis de Usuário

Os perfis previstos para a versão inicial do GTP são:

Perfil	Responsabilidade
Administrador Global	Administração completa da plataforma.
Administrador da Congregação	Administração da congregação e seus recursos.
Ancião	Gestão das atividades pastorais e operacionais autorizadas.
Servo Ministerial	Apoio às atividades administrativas.
Publicador	Execução das operações permitidas ao publicador.
Usuário de Consulta	Acesso somente para leitura.

A criação de novos perfis deverá ocorrer por meio de parametrização, preservando a estrutura arquitetural.

4.6 Estrutura de Permissões

As permissões serão definidas por funcionalidade e operação.

Exemplos:

Módulo	Permissões
Usuários	Criar, consultar, atualizar, desativar.
Territórios	Criar, distribuir, editar, recolher, arquivar.
Publicações	Registrar, editar, cancelar, consultar.
Relatórios	Visualizar, exportar.

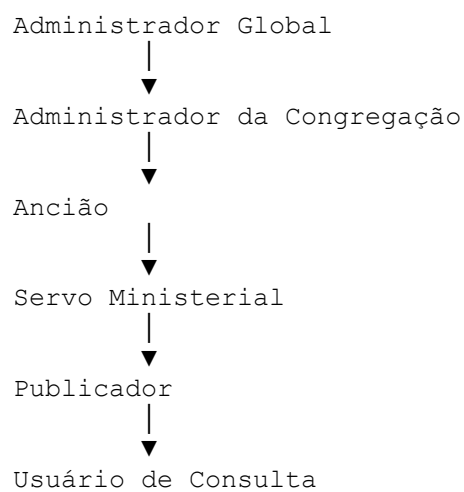
Módulo	Permissões
Configurações	Alterar parâmetros, gerenciar perfis.

Essa granularidade permite maior controle sobre as operações disponíveis para cada perfil.

4.7 Hierarquia de Perfis

A arquitetura permitirá estabelecer relações hierárquicas entre perfis quando necessário.

Exemplo conceitual:

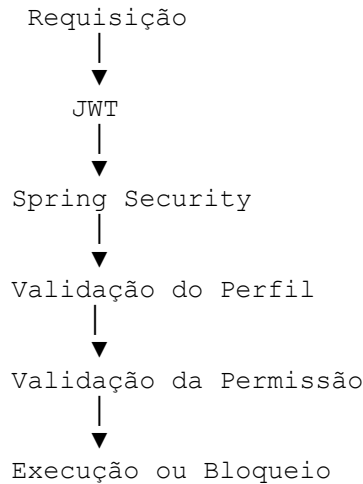


A herança de permissões deverá ser cuidadosamente avaliada para evitar concessões excessivas.

4.8 Controle de Acesso na API

Todos os endpoints protegidos deverão validar as permissões do usuário antes da execução.

Fluxo:



Caso o usuário não possua autorização, a API retornará **HTTP 403 (Forbidden)**.

4.9 Controle de Acesso no Frontend

O frontend utilizará as informações de perfil e permissões para adaptar a interface.

Exemplos:

- ocultação de menus;
- desabilitação de botões;
- restrição de páginas;
- bloqueio de operações.

Essas medidas melhoram a experiência do usuário, mas **não substituem** a validação obrigatória realizada pelo backend.

4.10 Autorização em Nível de Método

No backend, as regras de autorização poderão ser aplicadas diretamente sobre métodos da camada de aplicação.

Exemplos de verificações:

- acesso apenas para administradores;
- permissão para alterar territórios;
- autorização para exportar relatórios.

Essa abordagem aproxima as regras de segurança da lógica de negócio, reduzindo riscos de implementação.

4.11 Segregação de Funções (SoD)

Para reduzir riscos operacionais, determinadas operações poderão exigir perfis distintos.

Exemplos:

- quem cria um parâmetro crítico pode não ser o mesmo responsável por sua aprovação;
- determinadas ações administrativas podem exigir dupla validação em versões futuras.

A arquitetura permanecerá preparada para suportar essas evoluções.

4.12 Controle Contextual

Além do perfil, determinadas operações poderão considerar o contexto do usuário.

Exemplos:

- congregação ativa;
- vínculo com determinado território;
- estado da operação;
- período permitido para execução.

Essa abordagem amplia a precisão das decisões de autorização.

4.13 Negação por Padrão

Toda operação protegida seguirá a política de **negação por padrão**.

Isso significa que:

- recursos não explicitamente autorizados permanecerão inacessíveis;
- novas funcionalidades deverão declarar suas permissões antes de serem disponibilizadas.

Essa estratégia reduz riscos decorrentes de configurações incompletas.

4.14 Tratamento de Acesso Negado

Quando uma tentativa de acesso for bloqueada:

- a operação será interrompida;
- o evento poderá ser registrado para auditoria;
- o usuário receberá uma mensagem clara, sem exposição de detalhes internos.

Esse comportamento evita o vazamento de informações sobre a estrutura do sistema.

4.15 Auditoria das Decisões de Autorização

Eventos relevantes relacionados à autorização poderão ser registrados, como:

- acessos concedidos a operações críticas;
- tentativas de acesso negadas;
- alterações em perfis;
- modificações de permissões;
- elevação ou redução de privilégios.

Esses registros apoiam auditorias e investigações de incidentes.

4.16 Administração de Perfis e Permissões

A gestão de perfis e permissões será realizada por usuários autorizados.

As operações administrativas incluirão:

- criação de perfis;
- atualização de permissões;
- associação de usuários a perfis;
- revisão periódica dos acessos;
- desativação de perfis obsoletos.

Todas essas operações deverão ser auditadas.

4.17 Evolução da Arquitetura

A arquitetura foi projetada para permitir futuras ampliações, tais como:

- autenticação multifator (MFA);
- integração com provedores externos de identidade;
- políticas baseadas em atributos (ABAC);
- aprovação eletrônica de operações sensíveis;
- controle temporal de permissões.

Essas evoluções poderão ser incorporadas sem alterações estruturais significativas.

4.18 Boas Práticas

As seguintes diretrizes deverão ser observadas:

- centralizar decisões de autorização no backend;
- evitar regras duplicadas no frontend;
- revisar periodicamente os perfis existentes;
- documentar novas permissões;
- aplicar o princípio do menor privilégio;
- negar acesso por padrão;
- registrar operações críticas.

4.19 Relação com os Demais Documentos

Documento	Relação
Documento 03 – Regras de Negócio	Define as restrições funcionais de cada operação.
Documento 04 – Casos de Uso	Identifica quais atores executam cada caso de uso.
Documento 06 – Arquitetura Backend	Implementa a autorização utilizando Spring Security.
Documento 07 – Arquitetura Frontend	Adapta a interface conforme perfis e permissões.
Documento 10 – API REST	Especifica os requisitos de autorização dos endpoints.

5 – Segurança da API REST

5.1 Objetivo

Este capítulo define os mecanismos de segurança aplicados à API REST do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é proteger os serviços disponibilizados pela aplicação contra acessos não autorizados, manipulação indevida de dados, ataques conhecidos e uso abusivo da infraestrutura.

A API REST constitui o principal ponto de integração entre o frontend, aplicações futuras e serviços externos, sendo considerada um ativo crítico da arquitetura.

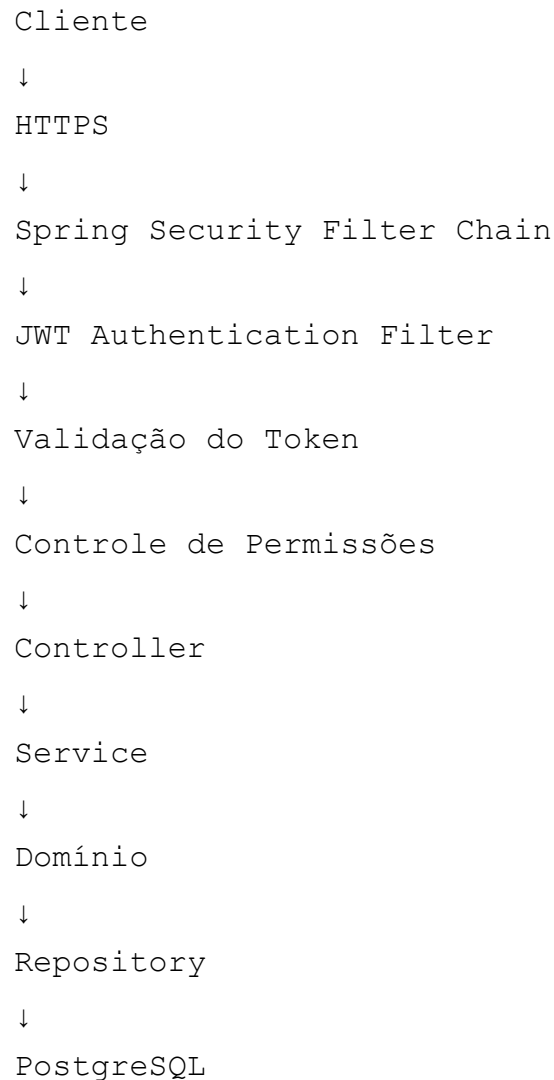
5.2 Princípios da Segurança da API

A proteção da API será baseada nos seguintes princípios:

- autenticação obrigatória;
- autorização em todas as operações protegidas;
- comunicação criptografada;
- validação completa das requisições;
- respostas padronizadas;
- menor privilégio;
- defesa em profundidade;
- rastreabilidade das operações.

5.3 Arquitetura da API Segura

Todas as requisições destinadas à API seguirão um fluxo controlado.



Cada etapa adiciona uma camada específica de proteção.

5.4 Comunicação Segura

Toda comunicação deverá ocorrer exclusivamente através de **HTTPS/TLS**.

Diretrizes:

- proibição de conexões HTTP em produção;
- certificados digitais válidos;

- criptografia ponta a ponta;
- proteção contra interceptação de tráfego.

Essa política garante confidencialidade e integridade das informações transmitidas.

5.5 Autenticação das Requisições

Toda requisição para recursos protegidos deverá apresentar um **JWT** válido.

Fluxo:

1. recebimento da requisição;
2. extração do token;
3. validação da assinatura;
4. verificação da expiração;
5. identificação do usuário;
6. carregamento de perfis e permissões.

Requisições sem autenticação ou com token inválido serão rejeitadas com **HTTP 401 (Unauthorized)**.

5.6 Autorização das Operações

Após a autenticação, o sistema verificará se o usuário possui autorização para executar a operação solicitada.

As verificações poderão considerar:

- perfil do usuário;
- permissões específicas;

- congregação ativa;
- contexto da operação;
- estado do recurso.

Caso a autorização falhe, será retornado **HTTP 403 (Forbidden)**.

5.7 Proteção dos Endpoints

Os endpoints serão classificados em diferentes níveis de acesso.

Categoria	Proteção
Públicos	Não exigem autenticação (ex.: login, recuperação de senha).
Autenticados	Exigem JWT válido.
Restritos	Exigem perfil específico.
Administrativos	Exigem privilégios elevados e auditoria reforçada.

Essa classificação facilita a configuração das políticas de segurança.

5.8 Cadeia de Filtros do Spring Security

A API utilizará uma cadeia de filtros responsável por aplicar as políticas de segurança antes da execução dos controladores.

Principais responsabilidades:

- identificar requisições;
- validar o JWT;
- autenticar o usuário;
- aplicar regras de autorização;

- registrar eventos relevantes;
- encaminhar ou bloquear a requisição.

5.9 Validação de Entrada

Todos os dados recebidos pela API serão validados.

As validações incluirão:

- obrigatoriedade de campos;
- tipos de dados;
- formatos válidos;
- limites de tamanho;
- valores permitidos;
- coerência entre informações.

Essas validações ocorrerão antes da execução das regras de negócio.

5.10 Proteção Contra Injeções

A arquitetura deverá reduzir os riscos associados a ataques de injeção.

Diretrizes:

- uso de consultas parametrizadas;
- utilização do JPA/Hibernate;
- validação de entradas;
- escape de caracteres quando necessário;
- rejeição de dados inválidos.

Essas práticas ajudam a mitigar ataques como SQL Injection.

5.11 Proteção Contra Enumeração de Recursos

A API evitará revelar informações que permitam inferir a existência de usuários, registros ou recursos protegidos.

Exemplos:

- mensagens genéricas para falhas de autenticação;
- respostas padronizadas para recursos inexistentes ou inacessíveis;
- ausência de detalhes internos em mensagens de erro.

5.12 Tratamento Padronizado de Erros

Todas as respostas de erro seguirão um formato consistente.

Campos sugeridos:

Campo	Descrição
--------------	------------------

timestamp	Data e hora da ocorrência.
-----------	----------------------------

status	Código HTTP.
--------	--------------

error	Identificação resumida do erro.
-------	---------------------------------

message	Mensagem amigável.
---------	--------------------

path	Recurso acessado.
------	-------------------

requestId	Identificador para rastreamento.
-----------	----------------------------------

Essa padronização facilita o diagnóstico e a integração com clientes da API.

5.13 CORS (Cross-Origin Resource Sharing)

A política de CORS será configurada explicitamente.

Diretrizes:

- permitir apenas origens autorizadas;
- restringir métodos HTTP quando necessário;
- limitar cabeçalhos aceitos;
- evitar configurações excessivamente permissivas.

Em ambientes de produção, o uso de "*" como origem deverá ser evitado.

5.14 Rate Limiting

A arquitetura deverá prever mecanismos de limitação de requisições para reduzir abusos.

Exemplos de aplicação:

- tentativas de login;
- consultas públicas;
- endpoints administrativos;
- operações de exportação.

Os limites poderão variar conforme o tipo de recurso e o perfil do usuário.

5.15 Proteção Contra Ataques de Repetição

Para reduzir o risco de reutilização indevida de requisições:

- tokens possuirão tempo de validade;
- operações críticas poderão exigir verificações adicionais;

- registros de auditoria permitirão identificar padrões anômalos.

5.16 Versionamento Seguro da API

A API deverá adotar versionamento explícito.

Exemplo:

```
/api/v1/territories  
/api/v1/publications  
/api/v1/users
```

Novas versões deverão coexistir temporariamente com as anteriores, reduzindo impactos sobre clientes já integrados.

5.17 Monitoramento da API

Eventos relevantes deverão ser monitorados.

Exemplos:

- falhas de autenticação;
- acessos negados;
- erros internos;
- aumento anormal de requisições;
- tentativas repetidas de acesso.

Essas informações apoiarão a identificação precoce de incidentes.

5.18 Auditoria das Requisições

Operações críticas poderão gerar registros contendo:

- usuário autenticado;

- recurso acessado;
- operação realizada;
- data e hora;
- resultado da execução;
- identificador da requisição.

Esses registros facilitarão auditorias e investigações.

5.19 Boas Práticas para APIs REST Seguras

As seguintes diretrizes deverão ser observadas:

- utilizar HTTPS em todos os ambientes produtivos;
- proteger todos os endpoints sensíveis;
- validar todas as entradas;
- aplicar autorização no backend;
- padronizar respostas;
- registrar operações críticas;
- evitar exposição de detalhes internos;
- manter dependências atualizadas.

5.20 Integração com o Frontend

O frontend será responsável por:

- anexar o JWT às requisições protegidas;
- tratar respostas **401** e **403**;
- renovar ou encerrar sessões conforme necessário;

- apresentar mensagens apropriadas ao usuário.

Toda decisão de segurança permanecerá sob responsabilidade do backend.

5.21 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Implementa a cadeia de filtros, autenticação e autorização.
Documento 07 – Arquitetura Frontend	Consome os serviços protegidos da API.
Documento 10 – API REST	Especifica os contratos, endpoints e modelos de dados expostos pela API.

6 – Segurança do Frontend

6.1 Objetivo

Este capítulo define a arquitetura de segurança do frontend do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é estabelecer diretrizes para proteger a interface da aplicação, controlar a navegação entre módulos, gerenciar a sessão do usuário e reduzir riscos associados a ataques direcionados ao navegador.

A segurança do frontend atua de forma complementar à segurança do backend, melhorando a experiência do usuário e reduzindo tentativas de exploração da interface, sem substituir as validações obrigatórias realizadas no servidor.

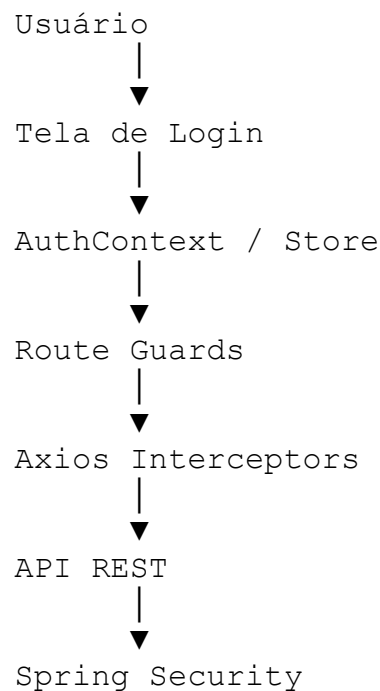
6.2 Princípios da Segurança do Frontend

A implementação seguirá os seguintes princípios:

- responsabilidade compartilhada entre frontend e backend;
- proteção da navegação;
- minimização da exposição de dados sensíveis;
- validação inicial das entradas;
- tratamento seguro de erros;
- defesa em profundidade;
- aderência às recomendações do **OWASP Top 10**.

6.3 Arquitetura Geral

A segurança será integrada aos principais componentes da aplicação React.



Cada componente possui responsabilidades específicas para manter a segurança da navegação e das requisições.

6.4 Proteção das Rotas

A aplicação distinguirá rotas públicas e protegidas.

Categorias:

Tipo	Exemplo
Pública	Login, recuperação de senha
Autenticada	Dashboard, territórios, publicações
Restrita	Administração, auditoria
Consulta	Relatórios somente leitura

Antes da renderização de uma rota protegida, o estado de autenticação será verificado.

6.5 Guards de Navegação

A navegação será protegida por componentes específicos (Route Guards).

Responsabilidades:

- verificar autenticação;
- validar permissões;
- impedir acesso indevido;
- redirecionar para login quando necessário;
- exibir página de acesso negado em caso de autorização insuficiente.

6.6 Gerenciamento da Sessão

A sessão do usuário será controlada de forma centralizada.

O gerenciamento incluirá:

- identificação do usuário autenticado;
- perfis e permissões;
- congregação ativa;
- estado da autenticação;
- expiração da sessão.

A implementação deverá evitar duplicação de estado entre diferentes componentes.

6.7 Armazenamento do JWT

O frontend utilizará o JWT apenas durante o tempo necessário para a operação da aplicação.

Diretrizes:

- evitar exposição desnecessária do token;
- impedir registro em logs do navegador;
- remover o token imediatamente após o logout;
- tratar corretamente tokens expirados.

A estratégia de armazenamento (memória, cookies seguros ou outro mecanismo) deverá ser definida considerando os requisitos de segurança e o ambiente de implantação.

6.8 Interceptadores HTTP

As requisições HTTP utilizarão interceptadores para padronizar comportamentos.

Responsabilidades:

- anexar o JWT às requisições protegidas;
- tratar respostas **401** e **403**;
- identificar expiração de sessão;
- registrar falhas relevantes;
- redirecionar o usuário quando necessário.

Essa centralização reduz duplicação de lógica e facilita a manutenção.

6.9 Validação de Entradas

Antes do envio ao backend, os dados serão validados.

Exemplos:

- obrigatoriedade;
- formato;
- tamanho;
- tipos de dados;
- consistência básica.

Essas validações têm como objetivo melhorar a experiência do usuário e reduzir requisições inválidas, sem substituir as validações do servidor.

6.10 Sanitização de Conteúdo

Sempre que a aplicação precisar exibir conteúdo dinâmico proveniente de fontes externas ou de entrada do usuário, esse conteúdo deverá ser tratado de forma segura.

Diretrizes:

- evitar inserção direta de HTML;
- utilizar componentes React para renderização;
- restringir o uso de mecanismos que permitam HTML arbitrário;
- sanitizar conteúdo quando a renderização de HTML for indispensável.

6.11 Proteção Contra Cross-Site Scripting (XSS)

A aplicação adotará medidas para reduzir riscos de XSS.

Boas práticas:

- escapar automaticamente conteúdos renderizados;
- evitar APIs que permitam HTML não confiável;
- validar e sanitizar entradas;
- utilizar políticas de segurança do navegador.

Essas medidas reduzem a possibilidade de execução de scripts maliciosos.

6.12 Proteção Contra CSRF

Quando a autenticação utilizar mecanismos baseados em cookies, deverão ser adotadas medidas adicionais contra **Cross-Site Request Forgery (CSRF)**.

Caso a arquitetura permaneça baseada em **JWT enviado no cabeçalho Authorization**, o risco de CSRF é reduzido, mas ainda deverão ser observadas boas práticas para todas as operações sensíveis.

6.13 Content Security Policy (CSP)

A aplicação deverá ser compatível com políticas de **Content Security Policy (CSP)**.

Objetivos:

- restringir fontes de scripts;
- limitar carregamento de recursos externos;
- reduzir riscos de XSS;
- controlar execução de conteúdo ativo.

A política poderá ser ajustada conforme o ambiente de implantação.

6.14 Tratamento Seguro de Erros

As mensagens apresentadas ao usuário deverão ser claras, porém sem revelar detalhes internos da aplicação.

Não deverão ser expostos:

- stack traces;
- nomes de classes;
- consultas SQL;
- caminhos internos;
- configurações da infraestrutura.

Erros técnicos completos deverão permanecer restritos aos mecanismos de monitoramento.

6.15 Proteção de Dados na Interface

A interface exibirá apenas as informações necessárias para cada operação.

Diretrizes:

- ocultar campos sem autorização;
- evitar exposição de identificadores internos quando desnecessário;
- limitar dados retornados pela API;
- respeitar o princípio do menor privilégio.

6.16 Logout Seguro

Ao encerrar a sessão:

- remover o estado de autenticação;
- limpar informações armazenadas na aplicação;
- invalidar caches relacionados ao usuário;
- redirecionar para a tela de login.

Essa sequência reduz riscos de reutilização indevida da sessão.

6.17 Controle de Estado

O gerenciamento de estado global deverá preservar o isolamento entre usuários.

Boas práticas:

- reinicializar o estado após logout;

- evitar persistência de informações sensíveis;
- separar estado de autenticação dos demais módulos.

6.18 Dependências e Bibliotecas

As bibliotecas utilizadas pelo frontend deverão ser monitoradas quanto a vulnerabilidades conhecidas.

Diretrizes:

- manter dependências atualizadas;
- remover pacotes obsoletos;
- acompanhar boletins de segurança;
- utilizar ferramentas de auditoria de dependências.

6.19 Observabilidade

Eventos relevantes poderão ser registrados para fins de monitoramento.

Exemplos:

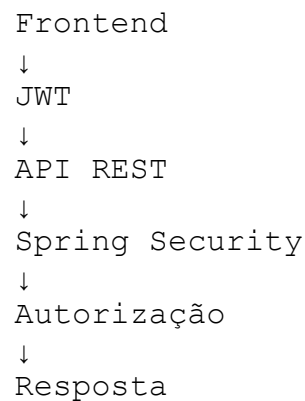
- falhas de autenticação;
- expiração de sessão;
- acessos negados;
- erros de comunicação com a API;
- exceções não tratadas.

Esses registros auxiliam na identificação e resposta a incidentes.

6.20 Integração com Backend

A segurança do frontend dependerá da integração contínua com os mecanismos implementados no backend.

Fluxo simplificado:



O backend continuará sendo a autoridade responsável pelas decisões de segurança.

6.21 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Implementa autenticação e autorização.
Documento 07 – Arquitetura Frontend	Define a estrutura da aplicação React e os componentes de navegação.
Documento 10 – API REST	Especifica os contratos consumidos pelo frontend e os requisitos de autenticação.

7 – Segurança do Backend

7.1 Objetivo

Este capítulo define a arquitetura de segurança do backend do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é estabelecer os mecanismos responsáveis por proteger a aplicação contra acessos não autorizados, garantir a integridade das regras de negócio, preservar a confidencialidade dos dados e assegurar que todas as operações sejam executadas de forma segura e auditável.

A implementação utilizará **Java 21**, **Spring Boot 3**, **Spring Security 6**, **JWT**, **Hibernate/JPA** e **PostgreSQL**, seguindo princípios de desenvolvimento seguro e arquitetura em camadas.

7.2 Princípios da Segurança do Backend

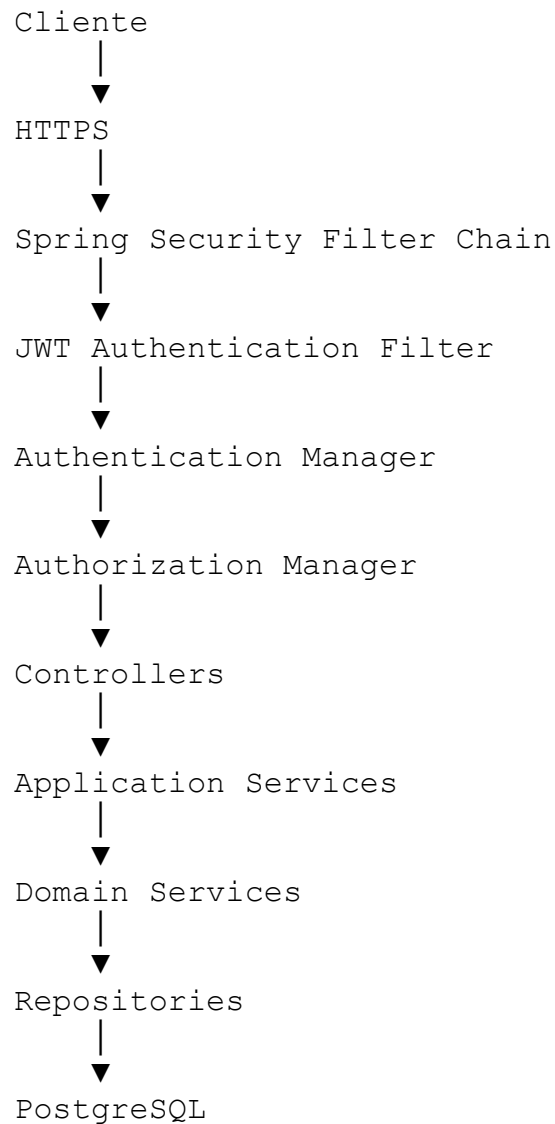
A segurança do backend será orientada pelos seguintes princípios:

- centralização das decisões de segurança;
- autenticação obrigatória para recursos protegidos;
- autorização baseada em papéis e permissões (RBAC);
- validação em todas as camadas;
- separação de responsabilidades;
- menor privilégio;
- defesa em profundidade;
- rastreabilidade das operações;
- tratamento seguro de falhas.

Esses princípios garantem consistência e reduzem riscos de implementação.

7.3 Arquitetura Geral da Segurança do Backend

A arquitetura de segurança será integrada ao fluxo de processamento das requisições.



Cada camada possui responsabilidades específicas e independentes.

7.4 Spring Security

O **Spring Security** será responsável por:

- autenticar usuários;

- validar tokens JWT;
- controlar autorização;
- proteger endpoints;
- gerenciar contexto de segurança;
- integrar filtros de autenticação.

Sua configuração será centralizada para facilitar manutenção e evolução.

7.5 Cadeia de Filtros (Security Filter Chain)

Todas as requisições passarão pela **Security Filter Chain** antes de alcançar os controladores.

Fluxo simplificado:

1. Recepção da requisição.
2. Identificação do endpoint solicitado.
3. Extração do token JWT.
4. Validação da assinatura e expiração.
5. Carregamento do usuário autenticado.
6. Verificação de perfis e permissões.
7. Encaminhamento para o controlador ou bloqueio da requisição.

Essa abordagem garante aplicação uniforme das políticas de segurança.

7.6 Gerenciamento de Usuários

O backend será responsável por:

- autenticar usuários;

- consultar informações de perfis;
- verificar permissões;
- controlar contas ativas e inativas;
- aplicar políticas de bloqueio quando necessário.

As informações de identidade serão obtidas a partir da base de dados oficial da aplicação.

7.7 Proteção de Senhas

As senhas serão armazenadas exclusivamente em formato criptografado.

Diretrizes:

- utilização do algoritmo **BCrypt**;
- geração automática de *salt*;
- fator de custo configurável;
- proibição de armazenamento em texto puro.

O backend jamais retornará ou registrará senhas em logs.

7.8 Validação das Regras de Negócio

As regras críticas de negócio serão executadas na camada de aplicação e no domínio.

Exemplos:

- impedir distribuição duplicada de territórios;
- validar datas de devolução;
- restringir alterações conforme o estado do recurso;

- impedir operações incompatíveis com o perfil do usuário.

Mesmo que o frontend realize validações preliminares, a verificação definitiva ocorrerá no backend.

7.9 Validação de Dados

Os dados recebidos pela API serão submetidos a validações utilizando **Jakarta Validation** e regras específicas da aplicação.

Entre as validações previstas:

- obrigatoriedade de campos;
- formatos válidos;
- limites de tamanho;
- intervalos numéricos;
- consistência entre informações relacionadas.

Requisições inválidas serão rejeitadas antes da execução da lógica de negócio.

7.10 Controle Transacional

Operações que modificam dados deverão ser executadas dentro de transações.

Objetivos:

- preservar consistência;
- evitar estados intermediários;
- garantir atomicidade;
- facilitar recuperação em caso de falha.

Em caso de erro, a transação será revertida automaticamente (*rollback*).

7.11 Segurança da Persistência

A camada de persistência seguirá boas práticas para proteção dos dados.

Diretrizes:

- uso de consultas parametrizadas;
- utilização do Hibernate/JPA;
- prevenção de SQL Injection;
- controle de integridade referencial;
- restrições definidas no banco de dados.

7.12 Tratamento Global de Exceções

As exceções serão tratadas de forma centralizada.

Benefícios:

- respostas padronizadas;
- ocultação de detalhes internos;
- simplificação do código;
- melhor rastreabilidade.

Informações técnicas detalhadas permanecerão restritas aos logs internos.

7.13 Proteção Contra Ataques Comuns

A arquitetura deverá mitigar riscos associados a:

- SQL Injection;
- Cross-Site Scripting (XSS) refletido em respostas;

- enumeração de usuários;
- força bruta;
- manipulação de parâmetros;
- acesso não autorizado;
- exploração de dependências vulneráveis.

As estratégias específicas serão detalhadas em capítulos posteriores.

7.14 Auditoria

Operações críticas deverão gerar registros contendo, quando aplicável:

- usuário autenticado;
- data e hora;
- endereço IP (quando disponível);
- recurso acessado;
- operação realizada;
- resultado da operação;
- identificador da requisição.

Esses registros apoiarão investigações e conformidade.

7.15 Observabilidade

O backend produzirá métricas e eventos relevantes para monitoramento.

Exemplos:

- autenticações bem-sucedidas;
- falhas de autenticação;

- erros de autorização;
- exceções não tratadas;
- tempo de resposta;
- operações críticas.

Essas informações poderão ser integradas a ferramentas de observabilidade.

7.16 Segurança das Configurações

As configurações sensíveis da aplicação deverão ser protegidas.

Incluem:

- segredos criptográficos;
- chaves de assinatura JWT;
- credenciais de banco de dados;
- parâmetros de autenticação;
- URLs de serviços externos.

Essas informações deverão ser fornecidas por variáveis de ambiente ou mecanismos equivalentes, evitando armazenamento em código-fonte.

7.17 Dependências e Atualizações

As bibliotecas utilizadas pelo backend deverão ser mantidas atualizadas.

Diretrizes:

- acompanhar vulnerabilidades conhecidas;
- aplicar correções de segurança;
- remover dependências obsoletas;

- revisar periodicamente componentes de terceiros.

7.18 Integração com Infraestrutura

A segurança do backend dependerá da infraestrutura de execução.

A integração abrangerá:

- HTTPS/TLS;
- Docker;
- PostgreSQL;
- mecanismos de backup;
- monitoramento;
- controle de acesso ao ambiente.

Esses aspectos serão detalhados nos capítulos de infraestrutura e Docker.

7.19 Boas Práticas

As seguintes diretrizes deverão ser observadas durante o desenvolvimento:

- manter regras de segurança centralizadas;
- validar todas as entradas;
- nunca confiar em dados provenientes do cliente;
- registrar eventos críticos;
- proteger segredos da aplicação;
- revisar periodicamente permissões;
- aplicar o princípio do menor privilégio;
- realizar testes de segurança.

7.20 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a organização geral da arquitetura.
Documento 06 – Arquitetura Backend	Especifica a estrutura das camadas e dos componentes do servidor.
Documento 07 – Arquitetura Frontend	Integra a interface com os mecanismos de autenticação e autorização.
Documento 09 – Docker	Define as práticas de segurança do ambiente de execução.
Documento 10 – API REST	Especifica os contratos e os requisitos de segurança dos serviços expostos.

8 – Criptografia e Proteção de Dados

8.1 Objetivo

Este capítulo estabelece as diretrizes para utilização de criptografia e proteção de dados no **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é garantir a confidencialidade, a integridade, a autenticidade e a disponibilidade das informações tratadas pelo sistema, protegendo dados sensíveis contra acessos não autorizados, alterações indevidas, vazamentos e perdas.

As diretrizes aqui definidas abrangem a proteção de dados em trânsito, em repouso e durante o processamento, alinhando-se às boas práticas internacionais de segurança da informação e à legislação brasileira aplicável.

8.2 Princípios da Proteção de Dados

A estratégia de proteção de dados será baseada nos seguintes princípios:

- confidencialidade;
- integridade;
- disponibilidade;
- autenticidade;
- rastreabilidade;
- minimização de dados;
- necessidade de acesso;
- privacidade desde a concepção (*Privacy by Design*);
- segurança desde a concepção (*Security by Design*).

Esses princípios deverão orientar todas as decisões relacionadas ao tratamento das informações.

8.3 Classificação das Informações

Os dados tratados pelo GTP serão classificados conforme seu nível de sensibilidade.

Classificação Descrição		Exemplos
Pública	Informações sem restrição de acesso.	Documentação pública, versão da aplicação.
Interna	Informações destinadas ao uso operacional.	Configurações gerais, parâmetros internos.

Classificação	Descrição	Exemplos
Restrita	Informações acessíveis apenas a usuários autorizados.	Dados de territórios, publicações e usuários.
Sensível	Informações que exigem proteção reforçada.	Credenciais, chaves criptográficas, tokens, registros de auditoria críticos.

Essa classificação orientará os mecanismos de proteção aplicados a cada tipo de informação.

8.4 Dados em Trânsito

Toda comunicação entre clientes, servidores e serviços externos deverá ocorrer por meio de conexões criptografadas.

Diretrizes:

- utilização obrigatória de **HTTPS**;
- adoção do protocolo **TLS** em versões consideradas seguras;
- certificados digitais válidos;
- rejeição de conexões não criptografadas em ambientes de produção.

Essa política reduz riscos de interceptação, alteração ou captura de informações durante a transmissão.

8.5 Dados em Repouso

As informações armazenadas deverão ser protegidas contra acesso indevido.

Medidas previstas:

- controle rigoroso de acesso ao banco de dados;
- proteção das credenciais de acesso;
- uso de criptografia quando necessário para dados de maior sensibilidade;
- realização de backups protegidos;
- restrição de acesso aos arquivos de configuração.

8.6 Proteção de Senhas

As senhas dos usuários não serão armazenadas em formato reversível.

Diretrizes:

- utilização do algoritmo **BCrypt**;
- geração automática de *salt*;
- fator de custo configurável;
- comparação segura durante o processo de autenticação.

Em nenhuma circunstância o sistema exibirá ou registrará senhas em texto claro.

8.7 Proteção de Tokens

Os tokens de autenticação representam credenciais temporárias e deverão receber tratamento equivalente ao de informações sensíveis.

Diretrizes:

- assinatura digital dos JWT;
- definição de tempo de expiração;
- transmissão apenas por HTTPS;
- remoção imediata após o encerramento da sessão;

- proibição de registro em logs.

8.8 Gerenciamento de Chaves Criptográficas

As chaves utilizadas para assinatura de tokens e outros mecanismos criptográficos deverão ser protegidas.

Boas práticas:

- armazenamento fora do código-fonte;
- utilização de variáveis de ambiente ou serviços especializados de gerenciamento de segredos;
- rotação periódica quando aplicável;
- acesso restrito a componentes autorizados.

8.9 Proteção de Segredos da Aplicação

Informações confidenciais da infraestrutura deverão ser tratadas como segredos.

Exemplos:

- chaves JWT;
- credenciais do banco de dados;
- credenciais de serviços externos;
- certificados digitais;
- tokens de integração.

Esses dados não deverão ser incluídos em repositórios públicos ou distribuídos juntamente com o código da aplicação.

8.10 Integridade dos Dados

A integridade das informações será garantida por meio de mecanismos técnicos e regras de negócio.

Entre as medidas adotadas:

- validações de domínio;
- controle transacional;
- restrições do banco de dados;
- auditoria de alterações;
- assinatura digital dos tokens.

8.11 Proteção de Dados Pessoais

O tratamento de dados pessoais deverá observar os princípios estabelecidos pela **Lei Geral de Proteção de Dados (LGPD)**.

Diretrizes:

- coleta apenas das informações necessárias;
- utilização compatível com a finalidade do sistema;
- controle de acesso baseado em perfis;
- proteção contra divulgação indevida;
- registro das operações relevantes.

Sempre que possível, deverão ser evitadas coletas desnecessárias de dados pessoais.

8.12 Privacidade desde a Concepção

A privacidade será considerada desde as fases iniciais do desenvolvimento.

Isso implica:

- definição prévia da finalidade de cada dado coletado;
- limitação do acesso às informações;
- revisão periódica dos dados armazenados;
- descarte seguro quando não houver necessidade de retenção.

8.13 Retenção e Descarte de Dados

As informações deverão permanecer armazenadas apenas pelo período necessário ao cumprimento de requisitos legais, operacionais ou históricos.

Após esse período:

- os dados poderão ser anonimizados;
- quando aplicável, deverão ser removidos de forma segura;
- os procedimentos deverão preservar a consistência da base de dados.

8.14 Backups Seguros

Os mecanismos de backup deverão garantir a recuperação das informações sem comprometer sua confidencialidade.

Diretrizes:

- backups periódicos;
- armazenamento protegido;
- controle de acesso;

- testes regulares de restauração;
- documentação dos procedimentos de recuperação.

8.15 Recuperação de Desastres

A arquitetura deverá permitir a recuperação do ambiente após falhas críticas.

O plano de recuperação deverá contemplar:

- restauração do banco de dados;
- recuperação das configurações da aplicação;
- reimplantação dos serviços;
- validação da integridade dos dados restaurados.

8.16 Criptografia em Integrações

Quando o GTP integrar-se a serviços externos, a comunicação deverá seguir padrões seguros.

Requisitos:

- uso de HTTPS/TLS;
- autenticação das partes envolvidas;
- validação de certificados;
- proteção das credenciais utilizadas na integração.

8.17 Auditoria de Acesso aos Dados

Os acessos a informações sensíveis poderão ser registrados para fins de auditoria.

Os registros poderão conter:

- usuário responsável;
- recurso acessado;
- data e hora;
- tipo de operação;
- resultado da operação.

Esses registros apoiarão investigações e comprovação de conformidade.

8.18 Boas Práticas de Proteção de Dados

As seguintes diretrizes deverão ser observadas:

- utilizar algoritmos criptográficos reconhecidos;
- proteger chaves e segredos da aplicação;
- evitar armazenamento de informações desnecessárias;
- limitar o acesso aos dados;
- revisar periodicamente as permissões;
- realizar backups protegidos;
- testar regularmente os mecanismos de recuperação.

8.19 Relação com os Demais Documentos

Documento	Relação
Documento 06 – Arquitetura Backend	Implementa os mecanismos de criptografia e proteção de dados no servidor.

Documento	Relação
Documento 07 – Arquitetura Frontend	Define o tratamento seguro das informações na interface do usuário.
Documento 09 – Docker	Estabelece práticas para proteção do ambiente de execução e dos segredos da aplicação.
Documento 10 – API REST	Especifica os requisitos de segurança para a transmissão de dados entre clientes e servidores.

9 – Auditoria e Logs

9.1 Objetivo

Este capítulo estabelece as diretrizes para auditoria, monitoramento e registro de eventos do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é garantir a rastreabilidade das operações realizadas na plataforma, permitindo identificar quem executou determinada ação, quando ocorreu, sobre qual recurso e qual foi o resultado da operação.

A arquitetura de auditoria foi projetada para apoiar a segurança da informação, a investigação de incidentes, a governança da aplicação e a melhoria contínua do sistema.

9.2 Princípios da Auditoria

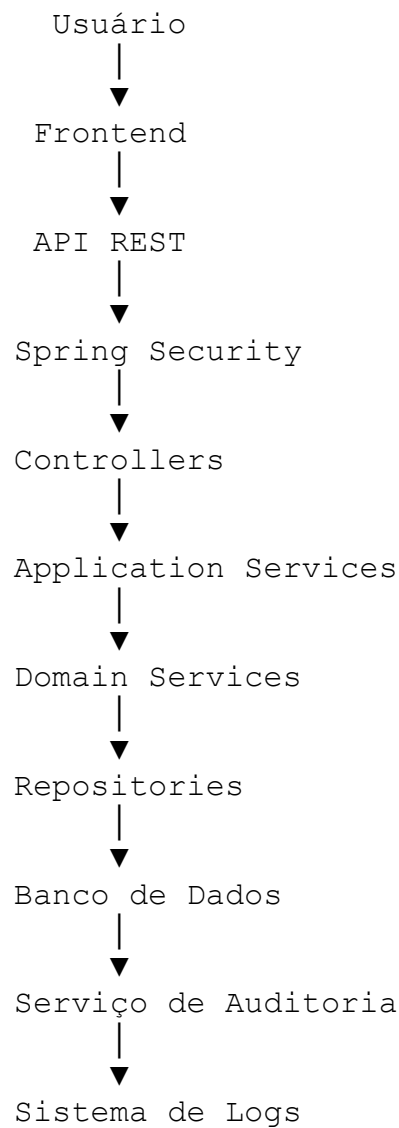
A auditoria do GTP será baseada nos seguintes princípios:

- rastreabilidade;
- integridade dos registros;
- confidencialidade das informações;
- disponibilidade dos logs;

- padronização;
- mínimo impacto sobre o desempenho da aplicação;
- suporte à investigação de incidentes;
- conformidade com a LGPD.

9.3 Arquitetura da Auditoria

A geração de registros ocorrerá ao longo de todo o fluxo de processamento da aplicação.



A auditoria será tratada como um serviço transversal, independente das regras de negócio.

9.4 Eventos Auditáveis

Os seguintes eventos deverão ser registrados:

Autenticação

- login realizado;
- logout;
- falhas de autenticação;
- bloqueio de conta;
- redefinição de senha;
- expiração de sessão.

Autorização

- acessos concedidos a operações críticas;
- acessos negados;
- alterações de perfis;
- alterações de permissões.

Operações de Negócio

- criação de registros;
- atualização de dados;
- exclusão lógica;
- arquivamento;
- restauração;
- distribuição de territórios;

- devolução de territórios;
- emissão de relatórios.

Administração

- alterações de configurações;
- parametrizações;
- manutenção do sistema;
- atualização de usuários.

Infraestrutura

- inicialização da aplicação;
- encerramento;
- indisponibilidades;
- erros críticos;
- integração com serviços externos.

9.5 Informações Registradas

Cada evento de auditoria poderá conter:

Campo	Descrição
Event ID	Identificador único do evento
Timestamp	Data e hora
Usuário	Usuário autenticado

Campo	Descrição
Perfil	Perfil utilizado
Congregação	Contexto da operação
Operação	Ação executada
Recurso	Objeto afetado
Resultado	Sucesso ou falha
Código HTTP	Quando aplicável
Request ID	Identificador da requisição
Endereço IP	Quando disponível
User-Agent	Cliente utilizado

Esses dados permitem rastrear integralmente uma operação.

9.6 Correlação de Requisições

Cada requisição receberá um identificador único (*Request ID*).

Esse identificador permitirá:

- acompanhar uma operação ponta a ponta;
- correlacionar logs distribuídos;
- facilitar investigações;
- integrar ferramentas de observabilidade.

9.7 Níveis de Severidade

Os registros serão classificados conforme sua criticidade.

Nível	Utilização
TRACE	Diagnóstico detalhado durante desenvolvimento.
DEBUG	Informações técnicas para depuração.
INFO	Operações normais da aplicação.
WARN	Situações anormais sem interrupção do serviço.
ERROR	Falhas que impediram a conclusão de uma operação.
FATAL (quando suportado)	Erros críticos que comprometem a aplicação.

A utilização de cada nível deverá seguir critérios consistentes.

9.8 Estrutura Padronizada dos Logs

Os registros deverão seguir um formato uniforme.

Campos recomendados:

- data e hora;
- nível;
- serviço;
- classe ou componente;
- mensagem;
- usuário;
- Request ID;
- exceção (quando aplicável).

A padronização facilita consultas e integração com ferramentas de monitoramento.

9.9 Proteção dos Logs

Os registros de auditoria deverão ser protegidos contra alteração ou remoção não autorizada.

Diretrizes:

- acesso restrito;
- controle de permissões;
- armazenamento seguro;
- cópias de segurança;
- integridade dos registros.

Informações sensíveis não deverão ser registradas em texto claro.

9.10 Dados que Não Devem Ser Registrados

Os seguintes dados não deverão constar nos logs:

- senhas;
- tokens JWT completos;
- chaves criptográficas;
- credenciais de banco de dados;
- segredos da aplicação;
- informações pessoais desnecessárias.

Quando necessário, identificadores poderão ser mascarados ou parcialmente ocultados.

9.11 Monitoramento

O sistema deverá produzir informações que permitam monitoramento contínuo.

Exemplos:

- aumento de falhas de autenticação;
- acessos negados em excesso;
- exceções recorrentes;
- degradação de desempenho;
- falhas de integração.

Esses indicadores poderão gerar alertas operacionais.

9.12 Alertas de Segurança

A arquitetura deverá permitir configuração de alertas para eventos relevantes.

Exemplos:

- múltiplas tentativas de login malsucedidas;
- elevação indevida de privilégios;
- alterações em configurações críticas;
- indisponibilidade de serviços;
- falhas sucessivas de integração.

Os alertas deverão ser proporcionais ao nível de criticidade do evento.

9.13 Retenção dos Registros

Os registros deverão ser mantidos conforme políticas institucionais e requisitos legais.

A política de retenção deverá definir:

- período mínimo de armazenamento;
- critérios de arquivamento;
- descarte seguro;
- preservação de registros relacionados a investigações.

9.14 Auditoria de Alterações

As modificações em informações críticas deverão registrar:

- valor anterior;
- novo valor;
- usuário responsável;
- data e hora;
- justificativa, quando aplicável.

Essa funcionalidade amplia a rastreabilidade das operações administrativas.

9.15 Integração com Observabilidade

A arquitetura permitirá integração com ferramentas de monitoramento e observabilidade.

Exemplos de funcionalidades:

- consolidação de logs;
- métricas operacionais;
- painéis de monitoramento;
- análise de tendências;

- investigação de incidentes.

A escolha das ferramentas dependerá da infraestrutura adotada.

9.16 Tratamento de Exceções

As exceções deverão ser registradas de forma padronizada.

Cada registro poderá incluir:

- tipo da exceção;
- mensagem técnica;
- componente responsável;
- Request ID;
- contexto da operação.

Detalhes internos permanecerão restritos aos administradores autorizados.

9.17 Auditoria de Segurança

Além das operações funcionais, deverão ser registrados eventos relacionados à segurança.

Exemplos:

- alteração de permissões;
- criação de administradores;
- bloqueio de contas;
- redefinição de credenciais;
- alterações em políticas de segurança.

9.18 Boas Práticas

Durante o desenvolvimento deverão ser observadas as seguintes diretrizes:

- registrar apenas informações relevantes;
- evitar excesso de registros redundantes;
- proteger dados sensíveis;
- utilizar níveis de severidade adequados;
- padronizar mensagens;
- revisar periodicamente a política de auditoria;
- monitorar continuamente eventos críticos.

9.19 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral que incorpora os serviços de auditoria.
Documento 06 – Arquitetura Backend	Implementa os componentes responsáveis pela geração de logs.
Documento 07 – Arquitetura Frontend	Registra eventos relevantes da interface e integra-se ao monitoramento.
Documento 09 – Docker	Define a estratégia de persistência, coleta e gerenciamento dos logs da infraestrutura.
Documento 10 – API REST	Especifica o uso de identificadores de requisição e o tratamento padronizado de erros.

10 – Testes de Segurança, Conformidade e Melhoria Contínua

10.1 Objetivo

Este capítulo estabelece a estratégia de testes de segurança, verificação de conformidade e melhoria contínua do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é garantir que os mecanismos de segurança implementados permaneçam eficazes ao longo da evolução do sistema, reduzindo riscos, identificando vulnerabilidades e assegurando conformidade com normas, boas práticas e requisitos definidos pela arquitetura da aplicação.

A segurança será tratada como um processo permanente, integrado ao desenvolvimento, à implantação e à operação do sistema.

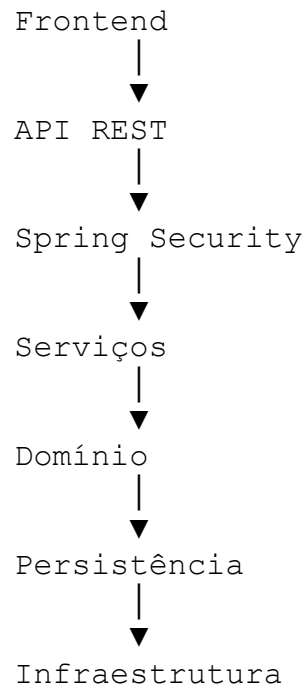
10.2 Princípios

A estratégia de segurança seguirá os seguintes princípios:

- prevenção;
- verificação contínua;
- automação sempre que possível;
- melhoria incremental;
- rastreabilidade;
- conformidade normativa;
- gestão de riscos;
- correção tempestiva de vulnerabilidades.

10.3 Estratégia de Testes

Os testes de segurança deverão abranger todas as camadas da aplicação.



A execução dos testes ocorrerá de forma integrada ao ciclo de desenvolvimento.

10.4 Testes de Autenticação

Os mecanismos de autenticação deverão ser verificados quanto aos seguintes aspectos:

- login válido;
- login com credenciais inválidas;
- usuário inexistente;
- conta bloqueada;
- conta inativa;
- senha incorreta;
- expiração de sessão;

- expiração de token;
- logout seguro;
- tentativa de reutilização de token.

Os resultados deverão ser registrados para análise.

10.5 Testes de Autorização

A validação da autorização deverá confirmar que:

- usuários acessam apenas recursos permitidos;
- perfis possuem as permissões corretas;
- operações críticas exigem autorização adequada;
- acessos indevidos retornam **HTTP 403 (Forbidden)**;
- novas funcionalidades respeitam o modelo RBAC.

10.6 Testes de Validação de Dados

As entradas da aplicação deverão ser testadas com diferentes cenários.

Exemplos:

- campos obrigatórios ausentes;
- formatos inválidos;
- valores fora dos limites;
- caracteres especiais;
- tentativas de injeção;
- dados inconsistentes.

Esses testes confirmam a robustez das validações implementadas.

10.7 Testes de APIs

Os endpoints REST deverão ser submetidos a testes específicos.

Itens avaliados:

- autenticação obrigatória;
- autorização por perfil;
- validação de parâmetros;
- respostas HTTP padronizadas;
- tratamento de erros;
- versionamento da API;
- políticas de CORS.

10.8 Testes de Vulnerabilidades

A aplicação deverá ser submetida periodicamente à identificação de vulnerabilidades.

Entre os aspectos avaliados:

- dependências desatualizadas;
- configurações inseguras;
- exposição de informações;
- autenticação inadequada;
- autorização incorreta;
- falhas de criptografia;
- problemas de configuração.

Sempre que possível, esses testes deverão ser automatizados.

10.9 Testes Baseados no OWASP Top 10

A estratégia de testes deverá contemplar os riscos mais relevantes identificados pelo **OWASP Top 10**, incluindo:

- falhas de controle de acesso;
- falhas criptográficas;
- ataques de injeção;
- configurações inseguras;
- componentes vulneráveis;
- falhas de autenticação;
- problemas de integridade;
- falhas de monitoramento e registro.

A lista de riscos deverá ser revisada periodicamente conforme as atualizações do projeto OWASP.

10.10 Testes Automatizados

A arquitetura incentivará a automação dos testes de segurança.

Exemplos:

- testes unitários;
- testes de integração;
- testes de autenticação;
- testes de autorização;
- validação de contratos da API;
- verificação de regras críticas.

A automação reduz regressões e aumenta a confiabilidade das entregas.

10.11 Revisão Segura de Código

Antes da integração de novas funcionalidades, o código deverá ser revisado.

Itens avaliados:

- aderência às diretrizes de segurança;
- tratamento de exceções;
- validação de entradas;
- proteção de dados sensíveis;
- uso correto de bibliotecas;
- conformidade com padrões arquiteturais.

10.12 Auditoria de Dependências

As bibliotecas utilizadas deverão ser monitoradas continuamente.

Boas práticas:

- identificação de vulnerabilidades conhecidas;
- atualização periódica;
- substituição de componentes obsoletos;
- acompanhamento de boletins de segurança.

10.13 Conformidade com a LGPD

A aplicação deverá observar os princípios da **Lei Geral de Proteção de Dados (LGPD)**.

Entre os aspectos considerados:

- minimização de dados;
- finalidade do tratamento;
- controle de acesso;
- proteção das informações;
- rastreabilidade;
- descarte seguro.

As práticas adotadas deverão ser revisadas periodicamente.

10.14 Gestão de Vulnerabilidades

O processo de gestão de vulnerabilidades compreenderá:

1. identificação;
2. classificação;
3. avaliação de impacto;
4. priorização;
5. correção;
6. validação da correção;
7. documentação.

Esse fluxo deverá ser incorporado ao processo de desenvolvimento da aplicação.

10.15 Monitoramento Contínuo

A segurança será acompanhada continuamente por meio de indicadores.

Exemplos:

- tentativas de login malsucedidas;
- acessos negados;
- falhas críticas;
- indisponibilidade de serviços;
- comportamento anômalo.

Esses indicadores apoiarão a tomada de decisões e a resposta a incidentes.

10.16 Plano de Resposta a Incidentes

A arquitetura deverá prever procedimentos para tratamento de incidentes de segurança.

O processo poderá incluir:

- identificação do incidente;
- contenção;
- investigação;
- correção;
- recuperação;
- documentação;
- revisão das causas;
- implementação de melhorias.

10.17 Capacitação da Equipe

A segurança depende também das práticas adotadas pela equipe de desenvolvimento.

Diretrizes:

- atualização constante sobre boas práticas;
- conhecimento dos padrões arquiteturais;
- revisão periódica das políticas de segurança;
- compartilhamento de lições aprendidas.

10.18 Indicadores de Segurança

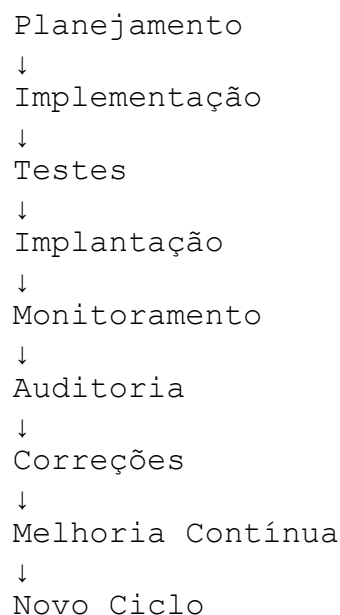
A gestão da segurança poderá acompanhar indicadores como:

Indicador	Objetivo
Taxa de falhas de autenticação	Identificar possíveis ataques.
Tempo médio de correção de vulnerabilidades	Avaliar a eficiência do processo.
Número de incidentes registrados	Medir a estabilidade da segurança.
Cobertura de testes de segurança	Avaliar a qualidade das verificações.
Dependências vulneráveis	Acompanhar riscos de terceiros.

Esses indicadores apoiam a melhoria contínua da plataforma.

10.19 Processo de Melhoria Contínua

A evolução da segurança seguirá um ciclo permanente.



Esse modelo favorece a adaptação constante da aplicação a novos riscos e requisitos.

10.20 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define os princípios arquiteturais que orientam os testes e a evolução da segurança.
Documento 06 – Arquitetura Backend	Implementa os mecanismos de segurança verificados neste capítulo.
Documento 07 – Arquitetura Frontend	Implementa os controles de segurança da interface e da navegação.
Documento 09 – Docker	Define a segurança do ambiente de execução e da infraestrutura.

Documento	Relação
Documento 10 – API REST	Especifica os contratos e requisitos de segurança que deverão ser validados pelos testes.

Conclusão Geral do Documento 08 – Segurança

O **Documento 08 – Segurança** define a arquitetura de segurança do **Gestor de Territórios e Publicações (GTP)** de forma abrangente, cobrindo desde a autenticação e autorização dos usuários até a proteção da infraestrutura, dos dados e dos processos operacionais. A adoção de **Spring Security 6**, **JWT**, **BCrypt**, **HTTPS/TLS**, políticas de controle de acesso baseadas em papéis (**RBAC**), criptografia, auditoria, monitoramento e práticas contínuas de verificação estabelece uma base sólida para o desenvolvimento de uma aplicação segura e escalável.

Além dos mecanismos técnicos, o documento incorpora princípios de **Security by Design**, **Privacy by Design**, conformidade com a **LGPD**, monitoramento contínuo e melhoria incremental, garantindo que a segurança seja tratada como um processo permanente e integrado ao ciclo de vida do software.

Com a conclusão deste documento, o GTP passa a contar com um conjunto consistente de diretrizes de segurança que servirá de referência para as fases de desenvolvimento, implantação, operação e evolução da plataforma, assegurando a proteção dos usuários, das informações e dos serviços oferecidos.